

Année	2025-2026	Type	Examen Session 1
Master	Informatique		
Code UE	4TIN705EX	Épreuve	Systèmes d'Exploitation
Date	12/12/2025	Documents	Non autorisés
Début	14h30	Durée	1h30

De nombreuses questions peuvent être traitées même si vous n'avez pas répondu aux précédentes. À la fin du sujet, vous trouverez un memento vous rappelant la syntaxe de quelques appels système utiles.

1 Questions de cours (échauffement)

Question 1 Les systèmes d'exploitation modernes utilisent la pagination en s'appuyant sur des tables de pages à plusieurs niveaux. Faites un joli dessin d'une table des pages à 3 niveaux, en détaillant le mécanisme de conversion d'adresses. Mentionnez le principal avantage d'un passage à une table à 4 niveaux (plutôt que 3). Y a-t-il une contrepartie ?

Question 2 Expliquez les grands principes de l'attaque Meltdown. Quelle est la caractéristique matérielle exploitée par cette attaque ? Décrivez la solution logicielle mise en place au sein des systèmes pour se prémunir contre cette attaque. Quel est le rapport avec Franz Beckenbauer ?

2 À l'aéroport

On souhaite simuler le fonctionnement d'un aéroport en modélisant le flux des passagers à leur sortie d'avion au moyen de *threads*. Leur nombre est aléatoire, tout comme le moment où chacun d'eux se décide à sortir de l'avion.

Chaque thread enchaîne l'exécution de `montrer_passeport` puis de `recuperer_bagage` puis se termine.

Question 1 Montrez que l'implémentation suivante de `montrer_passeport` risque d'aboutir à des erreurs.

```
bool guichet_dispo[NB_GUICHETS] = { TRUE, ..., TRUE }; // guichets de contrôle des passeports

void montrer_passeport ()
{
    int i;
    while (1)
        for (i = 0; i < NB_GUICHETS; i++)
            if (guichet_dispo [i] == TRUE) {
                guichet_dispo [i] = FALSE;
                break; // on suppose que ça provoque la sortie du while(1)
            }
    sleep (DELAI_EXAMEN_PASSEPORT);
    guichets_dispo [i] = TRUE; // le contrôle est terminé
}
```

Question 2 Corrigez le code en introduisant des moniteurs/conditions (et peut-être d'autres variables) partagés. Profitez de l'occasion pour éviter l'utilisation de boucles d'attente active tout en autorisant plusieurs contrôles simultanés.

Question 3 Une fois leur passeport vérifié, les passagers peuvent enfin aller flâner devant le tapis roulant qui leur apportera leur bagage tôt ou tard. Le tapis roulant est simplement implémenté par un tableau d'entiers de taille `MAX_BAGAGES` : chaque case contient soit `-1` si elle est vide, soit un numéro de bagage. En coulisse, un thread « bagagiste » s'affaire à déposer des bagages sur le tapis dès que c'est possible, tandis que les passagers scrutent le tapis avec anxiété...

```

int tapis[MAX_BAGAGES] = { -1, ..., -1 };
int charge = 0;

void recuperer_bagage (int mon_ticket)
{
    while (1) {
        while (charge == 0) ; // attendre

        for (int p = 0; p < MAX_BAGAGES; p++)
            // On admire le tapis qui tourne
            if (tapis [p] == mon_ticket) {
                // gagné... y'a plus qu'à attendre le bus !
                tapis [p] = -1;
                return;
            }
    }
}

void bagagiste ()
{
    while (1) {
        int bagage = recuperer_depuis_chariot();
        if (bagage == -1) return; // j'ai fini ma journée

        while (charge == MAX_BAGAGES) ; // attendre

        for (int p = 0; p < MAX_BAGAGES; p++)
            if (tapis [p] == -1) {
                // on a trouvé un emplacement, alors on y
                // jette le bagage de toutes ses forces
                tapis [p] = bagage;
                break;
            }
    }
}

```

Assurez la synchronisation correcte des threads en utilisant des moniteurs/conditions.

3 Gestion mémoire

On souhaite mettre en place l'allocation paresseuse de la mémoire accueillant le code des processus sous Nachos. L'idée est de n'allouer les pages de code qu'au moment où elles sont effectivement accédées pour la première fois. On suppose qu'en dehors du code, toutes les pages valides du processus sont allouées au lancement (pas d'allocation paresseuse généralisée).

Au sein de la classe `AddrSpace`, le segment de code est décrit par une structure nommée `codeVma` :

```

class AddrSpace: {
public:
    struct {
        int virtualAddr;    // adresse virtuelle de début
        int size;           // taille en octets
        int inFileOffset;   // offset dans le fichier exécutable
    } codeVma;
    ...
}

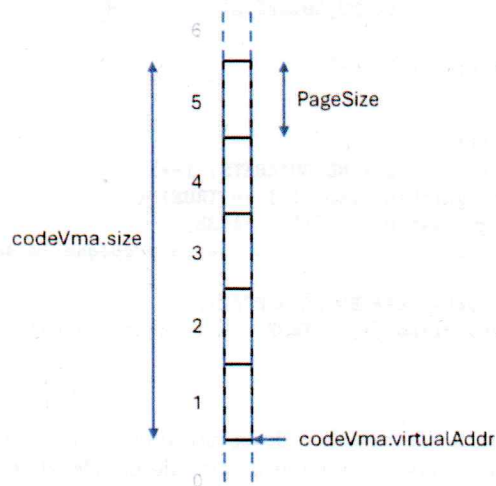
```

Le champ `virtualAddr` indique l'adresse virtuelle de début du segment de code, et le champ `size` sa taille en octets (voir figure ci-contre). Le champ `inFileOffset` indique la position dans le fichier exécutable où se trouve le code du processus. Dans le code d'origine de Nachos, lors de la création de l'espace d'adressage d'un processus, le code est chargé en mémoire au moyen d'un appel à la méthode `ReadAt` :

```

AddrSpace::AddrSpace (OpenFile *executable)
{
    ...
    // load code from executable file into memory
    executable->ReadAt (&(machine->mainMemory[codeVma.virtualAddr]),
                       codeVma.size, codeVma.inFileOffset);
    ...
}

```



(a) Illustration d'une pile de 5 pages réservée pour un thread. Dans cet exemple, seule les deux premières pages (grisées) sont allouées.

Question 1 Au début du devoir 3, il vous a été demandé de remplacer cet appel à `ReadAt` par un appel à une nouvelle méthode `ReadAtVirtual`. Pour quelle raison ?

Question 2 Écrivez une fonction `int inside_code (int virt_address)` qui renvoie 1 si l'adresse virtuelle passée en paramètre est comprise dans la région de code du processus courant , 0 sinon.

Question 3 On se place dorénavant dans le cadre de l'allocation paresseuse du code, c'est-à-dire qu'il n'est pas pré-chargé dans le constructeur de `AddrSpace` et les pages correspondantes sont marquées invalides dans la table des pages du processus. Lors du premier accès à l'une de ces pages, il faudra donc allouer une nouvelle page physique, et charger le contenu de la page depuis le fichier exécutable.

Expliquez pourquoi, dans le cadre de cette allocation paresseuse, l'utilisation de la méthode `ReadAt` sera suffisante et qu'il n'y aurait aucun intérêt à utiliser la méthode `ReadAtVirtual`.

Question 4 Lorsqu'un thread tente d'accéder à une page de code qui n'a pas encore été allouée, l'exception `PageFaultException` est levée, ce qui provoque le basculement dans le noyau dans le traitant `ExceptionHandler`.

```
void ExceptionHandler (ExceptionType which)
{
    if (which == PageFaultException) {
        int address = machine->ReadRegister (BadVAddrReg);
        ... // À compléter
    }
}
```

Complétez le code de manière à traiter correctement l'exception en allouant et chargeant, si nécessaire, la page de code manquante. Dans le cas contraire, `interrupt->PowerDown()` fera l'affaire.

On rappelle que les pages de code doivent être marquées en lecture seule. Ainsi, si une `PageFaultException` est levée à une adresse correspondant à une page de code déjà en place, c'est qu'une écriture a été tentée sur cette page.

Question 5 On suppose maintenant que les processus peuvent héberger plusieurs threads s'exécutant de manière concurrente. Ainsi, plusieurs threads d'un même processus peuvent simultanément déclencher des `PageFaultException` liées à l'allocation paresseuse du code.

Proposer une solution pour remédier aux problèmes pouvant survenir lorsque plusieurs threads déclenchent simultanément des `PageFaultException` pour la même page de code non encore allouée.

```
typedef ... mutex_t;
void mutex_lock(mutex_t *m);
void mutex_unlock(mutex_t *m);
```

Memento

```
typedef ... cond_t;
void cond_wait(cond_t *c, mutex_t *m);
void cond_signal(cond_t *c);
void cond_bcast(cond_t *c);
```